Task 01: Read CSV (Employees) + Filter High Salary Employees

employees.csv (example format)

```
id,name,salary,dept
1,Hemanth,50000,IT
2,Raj,30000,HR
3,Sita,70000,Finance
4,Manu,90000,IT
```

Python Code

```
import csv

high_salary_employees = []

with open("employees.csv", "r") as file:
    reader = csv.DictReader(file)

    for row in reader:
        salary = int(row["salary"])

        if salary >= 60000:
            high_salary_employees.append(row)

print("High Salary Employees:")
for emp in high_salary_employees:
    print(emp)
```

Task 02: Count Errors + Store Error Lines Separately

log.txt (example)

```
INFO User logged in
ERROR Invalid password
INFO Page loaded
ERROR Server timeout
INFO Logout successful
```

Python Code

```
error_lines = []
error_count = 0

with open("log.txt", "r") as file:
    for line in file:
        if "ERROR" in line:
            error_count += 1
            error_lines.append(line)
```

```
# Save errors separately
with open("errors.txt", "w") as file:
    file.writelines(error_lines)

print("Total Errors Found:", error_count)
print("Error lines saved in errors.txt")
```

Task 03: Generate Daily Report Automatically

Report Includes:

- Total employees
- High salary employees count
- Total errors
- Saves report into `daily_report.txt`

Python Code

```
import csv
from datetime import datetime

# ----- Employees Data -----
total_employees = 0
high_salary_count = 0

with open("employees.csv", "r") as file:
    reader = csv.DictReader(file)

    for row in reader:
        total_employees += 1
        if int(row["salary"]) >= 60000:
            high_salary_count += 1

# ----- Log Error Data -----
error_count = 0

with open("log.txt", "r") as file:
    for line in file:
        if "ERROR" in line:
            error_count += 1

# ----- Generate Report -----
today = datetime.now().strftime("%d-%m-%Y")

report = f"""
DAILY REPORT - {today}
-----------------------
Total Employees         : {total_employees}
High Salary Employees   : {high_salary_count}
Total Errors in Logs    : {error_count}
"""

with open("daily_report.txt", "w") as file:
    file.write(report)

print("Daily Report Generated Successfully!")
```

```
print("Saved as daily_report.txt")
```

Use Case: Student Marks File Reader + Average Calculator

Scenario:

- Read marks from a file (`marks.txt`)
- Calculate average marks
- Handle errors like:
  - File not found
  - Invalid data in file (letters instead of numbers)

---

marks.txt (Example File)

```
85
90
78
abc
60
```

Python Code (File Handling + Exception Handling)

```
def calculate_average_marks(filename):
    try:
        with open(filename, "r") as file:
            marks = []

            for line in file:
                line = line.strip()

                try:
                    marks.append(int(line))
                except ValueError:
                    print(f"Invalid mark found: {line} (Skipped)")

            if len(marks) == 0:
                return "No valid marks found!"

            average = sum(marks) / len(marks)
            return average

    except FileNotFoundError:
        return "Error: File not found!"
    except Exception as e:
        return f"Unexpected Error: {e}"


result = calculate_average_marks("marks.txt")
print("Average Marks:", result)
```

Simple Coding Questions

1) Handle ZeroDivisionError (Different Examples)

Example 1

```
try:
    a = 10
    b = 0
    print(a / b)
except ZeroDivisionError:
    print("Error: Cannot divide by zero")
```

Example 2 (User input)

```
try:
    a = int(input("Enter a number: "))
    b = int(input("Enter another number: "))
    print(a / b)
except ZeroDivisionError:
    print("Error: You entered 0 as divisor")
```

Example 3 (Inside function)

```
def divide(x, y):
    try:
        return x / y
    except ZeroDivisionError:
        return "Division by zero not allowed"

print(divide(50, 0))
```

---

2) Handle TypeError

Example: Adding number + string

```
try:
    result = 10 + "5"
    print(result)
except TypeError:
    print("TypeError: Cannot add int and string")
```

---

3) Handle AttributeError

Example: Using string methods on integer

```
try:
    x = 100
    print(x.upper())
```

```
except AttributeError:
    print("AttributeError: This object has no such attribute")
```

---

4) Validate User Input (Age, Salary)

Requirements:

- Age must be integer
- Salary must be float
- Age must be > 0
- Salary must be >= 0

```
try:
    age = int(input("Enter Age: "))
    salary = float(input("Enter Salary: "))

    if age <= 0:
        print("Invalid Age! Age must be greater than 0")
    elif salary < 0:
        print("Invalid Salary! Salary cannot be negative")
    else:
        print("Valid Input")
        print("Age:", age)
        print("Salary:", salary)

except ValueError:
    print("Invalid Input! Please enter correct numbers")
```

---

5) Custom Exception for ATM Withdrawal Rules

Rules:

- Amount must be multiple of 100
- Amount must be <= balance
- Amount must be > 0

```
class ATMError(Exception):
    pass


def withdraw(balance, amount):
    if amount <= 0:
        raise ATMError("Amount must be greater than 0")

    if amount % 100 != 0:
        raise ATMError("Amount must be a multiple of 100")

    if amount > balance:
        raise ATMError("Insufficient Balance")

    balance -= amount
    return balance
```

```
try:
    balance = 5000
    amount = int(input("Enter withdrawal amount: "))

    new_balance = withdraw(balance, amount)
    print("Withdrawal Successful!")
    print("Updated Balance:", new_balance)

except ATMError as e:
    print("ATM Withdrawal Failed:", e)

except ValueError:
    print("Invalid input! Enter numbers only.")
```

Project 1: Student Record Management System (student.txt)

Features

1. Add Student
2. View Students
3. Search Student by Name or ID

Full Python Code

```
FILENAME = "student.txt"


# ------------------ Add Student ------------------
def add_student():
    try:
        student_id = input("Enter Student ID: ")
        name = input("Enter Student Name: ")
        course = input("Enter Course: ")

        with open(FILENAME, "a") as file:
            file.write(f"{student_id},{name},{course}\n")

        print("Student Added Successfully!\n")

    except Exception as e:
        print("Error while adding student:", e)


# ------------------ View Students ------------------
def view_students():
    try:
        with open(FILENAME, "r") as file:
            data = file.readlines()

            if len(data) == 0:
```

```
                print("No student records found!\n")
                return

            print("\n--- Student Records ---")
            for line in data:
                student_id, name, course = line.strip().split(",")
                print(f"ID: {student_id} | Name: {name} | Course:
{course}")
            print()

    except FileNotFoundError:
        print("File not found! No records available.\n")

    except Exception as e:
        print("Error while viewing students:", e)


# ------------------ Search Students ------------------
def search_student():
    try:
        search_value = input("Enter Student ID or Name to Search:
").lower()

        with open(FILENAME, "r") as file:
            found = False

            for line in file:
                student_id, name, course = line.strip().split(",")

                if search_value == student_id.lower() or search_value ==
name.lower():
                    print("\nStudent Found!")
                    print(f"ID: {student_id} | Name: {name} | Course:
{course}\n")
                    found = True
                    break

            if not found:
                print("Student not found!\n")

    except FileNotFoundError:
        print("File not found! No records available.\n")

    except Exception as e:
        print("Error while searching student:", e)


# ------------------ Main Menu ------------------
def main():
    while True:
        print("===== Student Record Management =====")
        print("1. Add Student")
        print("2. View Students")
        print("3. Search Student")
        print("4. Exit")

        choice = input("Enter your choice: ")

        if choice == "1":
            add_student()
        elif choice == "2":
```

```
            view_students()
        elif choice == "3":
            search_student()
        elif choice == "4":
            print("Exiting... Thank you!")
            break
        else:
            print("Invalid Choice! Try again.\n")


main()
```

File Format (student.txt)

After adding students, the file will look like:

```
101,Hemanth,Python
102,Raj,Java
103,Sita,Data Science
```

Project 2: Word Counter Tool (summary.txt)

Features

✓ Count **Lines**
✓ Count **Words**
✓ Count **Characters**
✓ Handles exceptions properly

Full Python Code

```
FILENAME = "summary.txt"

def word_counter_tool():
    try:
        with open(FILENAME, "r") as file:
            content = file.read()

        # Counting
        lines = content.splitlines()
        words = content.split()
        characters = len(content)

        print("----- Word Counter Report -----")
        print("Total Lines      :", len(lines))
        print("Total Words      :", len(words))
        print("Total Characters :", characters)

    except FileNotFoundError:
```

```
        print("Error: summary.txt file not found!")

    except PermissionError:
        print("Error: Permission denied to read the file!")

    except Exception as e:
        print("Unexpected Error:", e)


word_counter_tool()
```

Exceptions Covered

FileNotFoundError

If `summary.txt` does not exist

PermissionError

If file is locked or no permission

General Exception

Catches unexpected errors safely

Project 3: Employee Salary Analyzer (employee.csv)

employee.csv (Example Format)

```
id,name,salary,dept
1,Hemanth,50000,IT
2,Raj,30000,HR
3,Sita,70000,Finance
4,Manu,90000,IT
```

Full Python Code

```
import csv

FILENAME = "employee.csv"

def salary_analyzer():
    try:
        salaries = []

        with open(FILENAME, "r") as file:
            reader = csv.DictReader(file)
```

```
            for row in reader:
                salaries.append(int(row["salary"]))

        if len(salaries) == 0:
            print("No salary data found in file!")
            return

        highest_salary = max(salaries)
        lowest_salary = min(salaries)
        average_salary = sum(salaries) / len(salaries)

        print("----- Employee Salary Report -----")
        print("Highest Salary :", highest_salary)
        print("Lowest Salary  :", lowest_salary)
        print("Average Salary :", average_salary)

    except FileNotFoundError:
        print("Error: employee.csv file not found!")

    except Exception as e:
        print("Unexpected Error:", e)


salary_analyzer()
```

Project 4: ATM Simulation (Python)

Code

```
class ATMError(Exception):
    pass


class ATM:
    def __init__(self, name, pin, balance=0):
        self.name = name
        self.pin = pin
        self.balance = balance

    # ---------------- Deposit ----------------
    def deposit(self, amount):
        if amount <= 0:
            raise ATMError("Deposit amount must be greater than 0")

        self.balance += amount
        return self.balance

    # ---------------- Withdraw ----------------
    def withdraw(self, amount):
        if amount <= 0:
            raise ATMError("Withdrawal amount must be greater than 0")

        if amount % 100 != 0:
            raise ATMError("Amount must be a multiple of 100")

        if amount > self.balance:
            raise ATMError("Insufficient Balance")
```

```
        self.balance -= amount
        return self.balance

    # ---------------- Check Balance ----------------
    def check_balance(self):
        return self.balance


# ---------------- Main Program ----------------
def main():
    user = ATM("Hemanth", 1234, 5000)

    print("===== Welcome to ATM Simulation =====")

    try:
        entered_pin = int(input("Enter PIN: "))

        if entered_pin != user.pin:
            print("Invalid PIN! Access Denied.")
            return

        while True:
            print("\n--- ATM MENU ---")
            print("1. Deposit")
            print("2. Withdraw")
            print("3. Check Balance")
            print("4. Exit")

            choice = input("Enter your choice: ")

            if choice == "1":
                amount = float(input("Enter deposit amount: "))
                try:
                    new_balance = user.deposit(amount)
                    print("Deposit Successful!")
                    print("Updated Balance:", new_balance)
                except ATMError as e:
                    print("Error:", e)

            elif choice == "2":
                amount = float(input("Enter withdrawal amount: "))
                try:
                    new_balance = user.withdraw(amount)
                    print("Withdrawal Successful!")
                    print("Updated Balance:", new_balance)
                except ATMError as e:
                    print("Error:", e)

            elif choice == "3":
                print("Current Balance:", user.check_balance())

            elif choice == "4":
                print("Thank you! Exiting ATM.")
                break

            else:
                print("Invalid choice! Try again.")

    except ValueError:
        print("Invalid input! PIN must be a number.")
```

```
main()
```

Project 5: User Login System (user.txt)

Features

1. Register User
2. Login User
3. View All Users
4. File Handling + Exception Handling

File Format (user.txt)

Each user will be stored like this:

```
username,password
```

Example:

```
hemanth,1234
raj,abcd
```

Python Code

```
FILENAME = "user.txt"


# ---------------- Register ----------------
def register_user():
    try:
        username = input("Enter new username: ")
        password = input("Enter new password: ")

        with open(FILENAME, "a") as file:
            file.write(f"{username},{password}\n")

        print("User Registered Successfully!\n")

    except Exception as e:
        print("Error while registering user:", e)


# ---------------- Login ----------------
def login_user():
    try:
        username_input = input("Enter username: ")
        password_input = input("Enter password: ")

        with open(FILENAME, "r") as file:
            for line in file:
                username, password = line.strip().split(",")
```

```
                if username_input == username and password_input ==
password:
                    print("Login Successful!\n")
                    return

        print("Invalid Username or Password!\n")

    except FileNotFoundError:
        print("Error: user.txt file not found! Please register first.\n")

    except Exception as e:
        print("Unexpected Error:", e)


# ---------------- View Users ----------------
def view_users():
    try:
        with open(FILENAME, "r") as file:
            data = file.readlines()

            if len(data) == 0:
                print("No users found!\n")
                return

            print("\n--- Registered Users ---")
            for line in data:
                username, password = line.strip().split(",")
                print("Username:", username)
            print()

    except FileNotFoundError:
        print("Error: user.txt file not found!\n")

    except Exception as e:
        print("Unexpected Error:", e)


# ---------------- Main Menu ----------------
def main():
    while True:
        print("===== User Login System =====")
        print("1. Register")
        print("2. Login")
        print("3. View Users")
        print("4. Exit")

        choice = input("Enter choice: ")

        if choice == "1":
            register_user()
        elif choice == "2":
            login_user()
        elif choice == "3":
            view_users()
        elif choice == "4":
            print("Exiting... Thank you!")
            break
        else:
            print("Invalid Choice! Try again.\n")
```

main()